第 6 章推導解答

DS06-01 • 搜尋次序與表示成本

BFS 為 A、B、C、D、E，距離分別 0、1、1、2、3。parent B=A、C=A、D=B、E=D，因 D 先由 B 發現。DFS 為 A、B、D、C、E，parent B=A、D=B、C=D、E=D。C 完成後回 D，再走 E。鄰接串列各點、邊處理常數次，完整搜尋 $\Theta(V+E)$；矩陣為每點掃 V 欄，$\Theta(V^2)$。

DS06-02 • 兩種 MST 算法

排序後先 BC=1，再 AB=2、DE=2、CD=3，已達四邊，總重 8。若繼續檢查，BD、AC、CE 的兩端皆已同分量，加入會成環。Prim 從 A 先 AB，再 BC，更新 C→D=3 優於 B→D=4，接著 CD、DE。以第一條 BC 為例，割 $\{B\}$ 與其餘頂點間最輕邊是 BC。若某 MST 不含 BC，加 BC 成環，環上另一條跨此割邊不輕於 1，替換後仍有不更重生成樹，因此 BC 安全。

DS06-03 • 負邊與負環的區分

無新增邊時不存在環，真實距離為 s=0、b=5、a=1、t=4。settled 版本先選 a=2，會過早固定，之後 b 的負邊無法修正 a，可能錯算 t=5。Bellman–Ford 在至多 V−1 輪傳播後得到正確距離。加入 a→b=1 形成 a→b→a 總重 −3 的可達負環，因此 a、b、t 可被任意降低。s 若沒有任何回到 s 的邊，$\delta(s,s)=0$ 仍有限。只有從負環可達的目的點受影響，不能整圖一概標成 $-\infty$。

DS06-04 • Floyd–Warshall 的迴圈順序

固定 k 時，更新將所有路徑分成不用 k 與使用 k，後者分解成 i→k 與 k→j，兩段的允許中介點皆來自前 k 個。在沒有相關負環時，最短路徑可選不重複頂點的路徑，這才支援上述分解。若改以 i、j 最外層，某個 D[i,j] 被處理時，所需的 D[i,k] 或 D[k,j] 未必已計算，不能視為完成同一 DP 層。反例為索引 0、1、2、3，邊 0→3、3→2、2→1，皆權 1。以 i,j,k 遞增且原地只跑一輪，計算 D[0,1] 時 D[0,2] 與 D[3,1] 尚為 ∞，會漏掉長 3 路徑。k 外層會先以 2 建立 3→1，再以 3 建立 0→1。

DS06-05 • AOE 關鍵路徑

E 為 s=0、a=3、b=2、t=8。L 逆推為 t=8、a=4、b=2、s=min(4−3,2−2)=0。活動 s→a 的鬆弛是 L[a]−3−E[s]=1，a→t 同為 1，另兩活動為 0。關鍵路徑 s→b→t 長 8。若 a 路兩活動各延長 1，該路由 7 變 9，總工期變 9。各自的總鬆弛是共享同一路徑的可用餘裕，不能獨立重複消耗。

第 6 章推導解答 （續）

DS06-06 • DAG 含負邊的單源最短路徑

先拓樸排序，令 d[s]=0、其餘 ∞，按拓樸序對有限 d[u] 的每條出邊鬆弛。處理 u 時，其所有可能前驅已處理完；任何 s→u 路徑的倒數第二點都是前驅，所以由歸納 d[u] 已最優。邊權可負，因沒有環，有限路徑不會無界下降。排序與每邊一次鬆弛合計 O(V+E)，空間 O(V) 不含輸入。有環時不存在完整拓樸序，前驅「先完成」的歸納基礎失效。

DS06-07 • 同圖三種目標與負邊初始化

加法Dijkstra在(s,p,q,t)依次取s、q、p、t：取s後(0,6,2,∞)，取q後(0,6,2,11)，取p後不改11；t.parent=q，最短路s,q,t總長11。瓶頸法起點−∞，取s後(−∞,6,2,∞)，取q後t=9、parent=q，取p後t=min(9,max(6,6))=6、parent=p；答案s,p,t的最大邊6。

Kruskal選s–q2、s–p6、p–t6，MST總重14。移除其s–t路徑最大邊6會切開s,t；若存在更輕跨割邊就能替換使MST更輕。所有s–t路徑都需跨此割，故瓶頸至少6，而該樹路徑達6。其總長12仍大於原圖最短11，證明兩個目標不同。

負邊反例只有s–t=−5，正確瓶頸−5；若B[s]=0，max(0,−5)=0錯誤。以−∞作空路徑max單位元才給−5。max延長永不降低，故minimax確定標籤證明仍成立；不能因允許負邊就把這個結論套回加法Dijkstra。

DS06-08 • 所有最優樹的必要邊與替代結構

權重1層的D–E是商圖唯一跨兩點邊，所以必選；收縮後分量由7降6。權重2的K4部分再把A,B,C及DE分量接成一個，需3邊，每條K4邊都位於cycle，皆可替代。權重5層把包含E的大分量與F,G接通，需2邊，三條皆可替代。任一MST總重1+3×2+2×5=17；只有D–E必選。各權值選邊數也由n−cλ門檻公式固定，不是由碰巧選的那棵推測。

取T1={DE,AB,BC,CD,EF,FG}，T2={DE,AC,AD,BD,EG,FG}。左側兩組分別為A–B–C–D與C–A–D–B路徑，右側各連通E,F,G，加DE後都連通無環且共6邊，因此兩者皆MST。共同邊DE、FG；對稱差含K4全部6條權2邊及EF、EG兩條權5邊，總重12+10=22>17。A在對稱差中degree3，所以它不可能是互不相交路徑／環（那些分量各點degree≤2）。不同最優樹的對稱差沒有matching所特有的degree限制。

第 6 章推導解答 （續）

原創驗算程式附錄: shortest_paths.cpp

```
#include <algorithm>
#include <functional>
#include <iostream>
#include <limits>
#include <queue>
#include <random>
#include <stdexcept>
#include <utility>
#include <vector>

using Distance = long long;
constexpr Distance infinity = std::numeric_limits<Distance>::max() / 4;

struct Edge {
	int from;
	int to;
	int weight;
};

struct Graph {
	int vertices;
	std::vector<Edge> edges;
};

void validate(const Graph& graph) {
	if (graph.vertices <= 0 || graph.vertices > 1000 || graph.edges.size() > 1000000) {
		throw std::invalid_argument("example supports 1..1000 vertices and at most
1000000 edges");
	}
	for (const auto& edge : graph.edges) {
		if (edge.from < 0 || edge.from >= graph.vertices || edge.to < 0 ||
			edge.to >= graph.vertices || edge.weight < -1000000 || edge.weight >
1000000) {
			throw std::invalid_argument("invalid endpoint or weight");
		}
	}
}

std::vector<Distance> dijkstra(const Graph& graph, int source) {
	validate(graph);
	if (source < 0 || source >= graph.vertices) {
		throw std::invalid_argument("invalid source");
	}
	std::vector<std::vector<std::pair<int, int>>> adjacent(graph.vertices);
	for (const auto& edge : graph.edges) {
		if (edge.weight < 0) {
			throw std::invalid_argument("Dijkstra requires nonnegative weights");
		}
```

第 6 章推導解答 （續）

```
                adjacent[edge.from].push_back({edge.to, edge.weight});
        }
        using Item = std::pair<Distance, int>;
        std::priority_queue<Item, std::vector<Item>, std::greater<Item>> queue;
        std::vector<Distance> distance(graph.vertices, infinity);
        distance[source] = 0;
        queue.push({0, source});
        while (!queue.empty()) {
                const auto [cost, vertex] = queue.top();
                queue.pop();
                if (cost != distance[vertex]) {
                        continue;
                }
                for (const auto& [next, weight] : adjacent[vertex]) {
                        const auto candidate = cost + weight;
                        if (candidate < distance[next]) {
                                distance[next] = candidate;
                                queue.push({candidate, next});
                        }
                }
        }
        return distance;
}

struct BellmanResult {
        std::vector<Distance> distance;
        bool reachable_negative_cycle;
};

BellmanResult bellman_ford(const Graph& graph, int source) {
        validate(graph);
        if (source < 0 || source >= graph.vertices) {
                throw std::invalid_argument("invalid source");
        }
        std::vector<Distance> distance(graph.vertices, infinity);
        distance[source] = 0;
        for (int round = 1; round < graph.vertices; ++round) {
                bool changed = false;
                for (const auto& edge : graph.edges) {
                        if (distance[edge.from] != infinity &&
                                distance[edge.from] + edge.weight < distance[edge.to]) {
                                distance[edge.to] = distance[edge.from] + edge.weight;
                                changed = true;
                        }
                }
                if (!changed) {
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

第 6 章推導解答 （續）

```
                        break;
                }
        }
        for (const auto& edge : graph.edges) {
                if (distance[edge.from] != infinity &&
                        distance[edge.from] + edge.weight < distance[edge.to]) {
                        return {distance, true};
                }
        }
        return {distance, false};
}

std::vector<std::vector<Distance>> floyd_warshall(const Graph& graph) {
        validate(graph);
        std::vector<std::vector<Distance>> distance(graph.vertices,
                std::vector<Distance>(graph.vertices, infinity));
        for (int vertex = 0; vertex < graph.vertices; ++vertex) {
                distance[vertex][vertex] = 0;
        }
        for (const auto& edge : graph.edges) {
                if (edge.weight < 0) {
                        throw std::invalid_argument("this Floyd oracle supports nonnegative
graphs only");
                }
                distance[edge.from][edge.to] = std::min(distance[edge.from][edge.to],
                        static_cast<Distance>(edge.weight));
        }
        for (int middle = 0; middle < graph.vertices; ++middle) {
                for (int from = 0; from < graph.vertices; ++from) {
                        for (int to = 0; to < graph.vertices; ++to) {
                                if (distance[from][middle] != infinity && distance[middle][to] !=
infinity) {
                                        distance[from][to] = std::min(distance[from][to],
                                                distance[from][middle] + distance[middle][to]);
                                }
                        }
                }
        }
        return distance;
}

void require(bool condition, const char* message) {
        if (!condition) {
                throw std::runtime_error(message);
        }
}

int main() {
```

第 6 章推導解答 （續）

```
	const Graph example{4, {{0, 1, 4}, {0, 2, 1}, {2, 1, 2}, {1, 3, 1}, {2, 3, 6}}};
	require(dijkstra(example, 0) == std::vector<Distance>({0, 3, 1, 4}),
		"slide distances differ");
	const Graph negative{4, {{0, 1, 2}, {0, 2, 5}, {2, 1, -4}, {1, 3, 3}}};
	const auto result = bellman_ford(negative, 0);
	require(!result.reachable_negative_cycle &&
		result.distance == std::vector<Distance>({0, 1, 5, 4}), "negative edge result
differs");
	bool rejected = false;
	try {
		dijkstra(negative, 0);
	} catch (const std::invalid_argument&) {
		rejected = true;
	}
	require(rejected, "Dijkstra accepted negative edge");
	Graph cycle = negative;
	cycle.edges.push_back({1, 2, 1});
	require(bellman_ford(cycle, 0).reachable_negative_cycle, "reachable negative cycle
missed");
	const Graph disconnected_cycle{4, {{0, 1, 3}, {2, 3, -2}, {3, 2, 1}}};
	require(!bellman_ford(disconnected_cycle, 0).reachable_negative_cycle,
		"unreachable negative cycle was misreported as reachable");
	std::mt19937 random(20261024);
	int checked = 0;
	for (int trial = 0; trial < 80; ++trial) {
		const int size = 1 + static_cast<int>(random() % 12);
		Graph graph{size, {}};
		for (int from = 0; from < size; ++from) {
			for (int to = 0; to < size; ++to) {
				if (from != to && random() % 4 == 0) {
					graph.edges.push_back({from, to, static_cast<int>(random()
% 20)});
				}
			}
		}
		const auto all = floyd_warshall(graph);
		for (int source = 0; source < size; ++source) {
			const auto single = dijkstra(graph, source);
			const auto bellman = bellman_ford(graph, source);
			require(!bellman.reachable_negative_cycle && single == all[source] &&
				single == bellman.distance, "shortest path algorithms disagree");
			++checked;
		}
	}
	std::cout << "PASS shortest paths: " << checked
		<< " source cases, exact trace, negative edge/cycle boundaries\n";
}
```

參考資料與署名

Ellis Horowitz, Sartaj Sahni, Dinesh Mehta. Fundamentals of Data Structures in C++, 2nd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。
https://github.com/xinhi6716/computer-science-self-study